Claude Ground-Truth Answer Key

BERT - 55-question deep-understanding benchmark. Answers extracted from the supplied Claude benchmark; no corrections added.

Q01

BERT stands for Bidirectional Encoder Representations from Transformers, a language representation model that pre-trains deep bidirectional representations from unlabeled text.

Q02

BERT_BASE has L=12 Transformer layers, H=768 hidden size, A=12 self-attention heads, and 110M total parameters.

Q03

BERT is pre-trained using two unsupervised tasks: (1) the Masked Language Model (MLM), which masks and predicts random tokens, and (2) Next Sentence Prediction (NSP), a binarized task that predicts whether one sentence follows another.

Q04

The pre-training corpus consists of BooksCorpus (800M words) and English Wikipedia (2,500M words); for Wikipedia only the text passages were used, ignoring lists, tables, and headers.

Q05

The paper reports in a footnote that the final model achieves 97%–98% accuracy on the Next Sentence Prediction task.

Q06

The MLM objective randomly masks some percentage of the input tokens and then trains the model to predict those masked tokens from their surrounding context, with the final hidden vectors at the mask positions fed into an output softmax over the vocabulary, as in a standard language model. The authors note this procedure is often referred to in the literature as a Cloze task (citing Taylor, 1953), because it is fundamentally a fill-in-the-blank prediction task.

Q07

"Bidirectional" means BERT's Transformer encoder uses self-attention in which every token can attend to all other tokens in the sequence — both before and after it — at every layer, so representations are built by jointly conditioning on left and right context simultaneously. This contrasts with unidirectional (left-to-right) models such as OpenAI GPT, where the self-attention is constrained so each token can only attend to tokens to its left, and with ELMo's approach of shallowly concatenating two separately trained left-to-right and right-to-left models rather than jointly conditioning on both directions within a single deep model.

Q08

In the feature-based approach (exemplified by ELMo), task-specific architectures are built that incorporate the pre-trained representations as additional input features, while the pre-trained parameters themselves stay fixed. In the fine-tuning approach (exemplified by OpenAI GPT), only minimal task-specific parameters are introduced, and the model is adapted to downstream tasks by training (fine-tuning) all of the pre-trained parameters together with the small number of new parameters.

Q09

When a sequence packs two sentences together, BERT differentiates them in two complementary ways: a special [SEP] token is inserted between the sentences as an explicit boundary marker, and a learned segment embedding is additionally added to every token in the sequence to indicate whether that token belongs to sentence A or sentence B. The final token input representation is the sum of the token, segment, and position embeddings, so the segment signal is present at every position, not just at the boundary.

Q10

NSP is a binarized task in which, for each pre-training example, sentence B is the true next sentence following A fifty percent of the time (labeled IsNext) and a random sentence from the corpus the other fifty percent of the time (labeled NotNext). The task is designed to pre-train the model to understand relationships between sentence pairs, which is useful for downstream tasks like QA and NLI that are not directly captured by language modeling alone. The final hidden vector C, corresponding to the [CLS] token, is used to make the IsNext/NotNext prediction. The paper notes elsewhere that C is not a meaningful sentence representation on its own until the model has been fine-tuned, since it was trained specifically for the NSP objective.

Q11

The authors argue that current pre-training techniques, especially fine-tuning-based ones, restrict the power of pre-trained representations because standard language models are unidirectional — this limits which architectures can be used during pre-training, for example forcing left-to-right attention as in OpenAI GPT.

Q12

The authors reason that restricting architecture choices to unidirectional attention is sub-optimal generally, but the harm is greatest for token-level tasks like question answering because it is crucial there to incorporate context from both directions around each token — a left-to-right constrained model (like GPT, where every token can only attend to preceding tokens) cannot use information appearing after a given token when building that token's representation, which is a much more severe handicap for fine-grained, per-token prediction than for tasks that only need a single holistic sentence-level judgment.

Q13

The paper explains that standard conditional language models can only be trained left-to-right or right-to-left, because if bidirectional conditioning were used naively, each word would be able to indirectly "see itself" through the network's multiple layers — the model could then trivially predict the target word from its own future context rather than genuinely learning to predict it, which is why BERT instead uses masking (predicting only a masked subset of tokens) rather than a normal bidirectional conditional language-modeling objective.

Q14

The authors note that many important downstream tasks, such as Question Answering and Natural Language Inference, are based on understanding the relationship between two sentences — something that is not directly captured by a language-modeling objective alone. To give the model this capability, they pre-train it on a binarized Next Sentence Prediction task that can be trivially generated from any monolingual corpus.

Q15

The authors state that it is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark, in order to be able to extract long contiguous sequences of text — implying that a shuffled, sentence-level corpus would break the coherent multi-sentence context that BERT's pre-training tasks (especially NSP, which depends on real sentence adjacency) rely on.

Q16

The training data generator first randomly chooses 15% of token positions in a sequence for prediction. For each chosen position, the token is (1) replaced with the literal [MASK] symbol 80% of the time, (2) replaced with a random token from the vocabulary 10% of the time, or (3) left unchanged 10% of the time. Regardless of which of the three happened, the model's final hidden vector at that position, T_i, is used to predict the original (true) token via cross-entropy loss.

Q17

Always replacing with [MASK] would create a downside: a mismatch between pre-training and fine-tuning, since the [MASK] token never actually appears during fine-tuning. To mitigate this, the procedure sometimes keeps the original token or substitutes a random one instead. The appendix adds that this also has a further benefit: because the Transformer encoder does not know in advance which positions it will be asked to predict, or which tokens have secretly been replaced with random words, it is forced to maintain a distributional contextual representation of every input token, not just the masked ones.

Q18

For a given token, its input representation is constructed by summing the corresponding token embedding, segment embedding, and position embedding, as visualized in Figure 2.

Q19

During fine-tuning only, BERT introduces a start vector S ∈ R^H and an end vector E ∈ R^H. The probability that word i is the start of the answer span is computed as the dot product of the token's final hidden vector T_i with S, passed through a softmax normalized over all words in the paragraph; the same style of formula, using E, gives the probability that a word is the end of the span. The score of a candidate span from position i to j is S·T_i + E·T_j, and the maximum-scoring span with j ≥ i is used as the prediction. The training objective is the sum of the log-likelihoods of the correct start and end positions.

Q20

SQuAD v2.0 extends v1.1 by allowing questions with no short answer in the passage. BERT handles this by treating questions without an answer as having an answer span whose start and end are both at the [CLS] token, extending the probability space for start/end positions to include that position. At prediction time, the score of the "no-answer" span, s_null = S·C + E·C, is compared to the score of the best non-null span; a non-null answer is predicted only when the best non-null span's score exceeds s_null by more than a threshold τ, where τ is selected on the dev set to maximize F1.

Q21

For SWAG, four separate input sequences are constructed per example, each the concatenation of the given sentence (as sentence A) with one of the four candidate continuations (as sentence B). The only new parameter is a single vector whose dot product with each sequence's [CLS] representation C yields a score for that choice; the four scores are normalized with a softmax layer to pick the most plausible continuation. This is structurally different from single-sentence classification tasks such as SST-2, which use one input sequence per example and a standard classification layer (weight matrix W) applied once to C to produce a probability distribution over the fixed label set.

Q22

The paper notes that a common prior pattern for text-pair tasks was to independently encode each text before applying a separate bidirectional cross-attention mechanism between them (citing Parikh et al. 2016 and Seo et al. 2017). BERT instead feeds the concatenated sentence pair through the same self-attention mechanism used for everything else; because self-attention over the full concatenated sequence lets every token attend to every other token regardless of which sentence it came from, this single encoding step effectively already includes bidirectional cross-attention between the two sentences, eliminating the need for a separate cross-attention stage.

Q23

Pre-training used the Adam optimizer with a learning rate of 1e-4, β1 = 0.9, β2 = 0.999, L2 weight decay of 0.01, learning-rate warmup over the first 10,000 steps, and linear decay of the learning rate thereafter.

Q24

To speed up pre-training, the model was pre-trained with a sequence length of 128 for 90% of the steps, then trained on the remaining 10% of steps with a sequence length of 512, which was done to learn the positional embeddings for longer sequences.

Q25

BERT uses a GELU activation (citing Hendrycks and Gimpel, 2016) rather than the standard ReLU activation, a choice made to follow OpenAI GPT.

Q26

The model was trained with a batch size of 256 sequences (256 sequences × 512 tokens = 128,000 tokens per batch) for 1,000,000 steps, which the paper states is approximately 40 epochs over the 3.3 billion word corpus (the combined size of BooksCorpus and Wikipedia).

Q27

In Table 1, BERT_LARGE's Average is 82.1 and OpenAI GPT's Average is 75.1, giving a difference of 7.0 points — not 7.7. The paper explains this discrepancy itself: Table 1's "Average" column is explicitly noted to be "slightly different from the official GLUE score" because the problematic WNLI set is excluded. The 7.7-point figure in the abstract instead corresponds to the official GLUE leaderboard scores discussed later in Section 4.1: BERT_LARGE scores 80.5 on the official leaderboard, versus OpenAI GPT's 72.8, giving 80.5 – 72.8 = 7.7, which matches the abstract exactly.

Q28

Computing BERT_BASE minus OpenAI GPT for each task in Table 1: MNLI-m +2.5, MNLI-mm +2.0, QQP +0.9, QNLI +3.1, SST-2 +2.2, CoLA +6.7, STS-B +5.8, MRPC +6.6, and RTE +10.4 (66.4 – 56.0). RTE shows the largest single-task improvement, at 10.4 points.

Q29

From Table 2, BERT_LARGE (Sgl.+TriviaQA) has a Test F1 of 91.8. The two "Top Leaderboard Systems" listed are #1 Ensemble – nlnet (Test F1 91.7) and #2 Ensemble – QANet (Test F1 90.5). 91.8 – 91.7 = 0.1 (against nlnet), while 91.8 – 90.5 = 1.3 (against QANet). Only the comparison against the #2 system (QANet) produces the +1.3 figure the text reports for the single-model comparison — the ensemble comparison (+1.5 F1) instead uses BERT_LARGE (Ens.+TriviaQA)'s Test F1 of 93.2 against the #1 system, nlnet's 91.7 (93.2 – 91.7 = 1.5).

Q30

From Table 3, BERT_LARGE (Single) achieves a Test F1 of 83.1. The highest Test F1 among the prior (non-BERT, non-human) systems shown is #1 Single – MIR-MRC (F-Net) at 78.0 — higher than #2 Single – nlnet (77.1) and the ensemble system unet (74.9). 83.1 – 78.0 = 5.1, matching the paper's stated improvement exactly.

Q31

From Table 4, BERT_LARGE's SWAG Test accuracy is 86.3%. ESIM+ELMo's Test accuracy is 59.2%, giving 86.3 – 59.2 = 27.1 points, matching the paper's "+27.1%" claim. OpenAI GPT's Test accuracy is 78.0%, giving 86.3 – 78.0 = 8.3 points, matching the paper's "8.3%" claim. Both stated improvements check out exactly against the table.

Q32

In Table 6, the smallest configuration (#L=3, #H=768, #A=12) has MNLI-m Dev accuracy of 77.9 and masked-LM perplexity of 5.84; the largest configuration (#L=24, #H=1024, #A=16) has MNLI-m Dev accuracy of 86.6 and perplexity of 3.23. Accuracy increases by 86.6 – 77.9 = 8.7 points as the model grows, while perplexity decreases (improves) by 5.84 – 3.23 = 2.61. Both trends move in the direction the authors describe as "strict accuracy improvement" with scale, and lower held-out perplexity, as model size increases.

Q33

From Table 5: "No NSP" MRPC accuracy is 86.5 and "LTR & No NSP" MRPC accuracy is 77.5, a drop of 86.5 – 77.5 = 9.0 points. BERT_BASE's SQuAD F1 is 88.5 and "LTR & No NSP"'s SQuAD F1 is 77.8, a drop of 88.5 – 77.8 = 10.7 points. The SQuAD (token-level) drop of 10.7 points is larger than the MRPC (sentence-level) drop of 9.0 points, by 10.7 – 9.0 = 1.7 points.

Q34

Both "No NSP" and "LTR & No NSP" are trained without the Next Sentence Prediction task, so the only difference between them is that "No NSP" still uses the bidirectional Masked LM objective, while "LTR & No NSP" is trained as a standard left-to-right language model instead. This comparison therefore isolates the effect of bidirectional representation training specifically. The paper reports that the LTR model performs worse than the MLM model on every task shown, with especially large drops on MRPC and SQuAD, demonstrating that deep bidirectionality itself — not merely the presence of NSP — is a major driver of BERT's performance.

Q35

Since a left-to-right model is intuitively expected to perform poorly on token-level prediction (its token-level hidden states have no right-side context), the authors added a randomly initialized BiLSTM on top of the "LTR & No NSP" model as a good-faith attempt to strengthen it, particularly for SQuAD. The BiLSTM did significantly improve SQuAD results relative to the plain LTR model, but the results were still far worse than those of the pre-trained bidirectional

MLM models. Additionally, the BiLSTM addition hurt performance on the GLUE tasks rather than helping.

Q36

In the feature-based section of Table 7, using only the non-contextual input embeddings gives a Dev F1 of 91.0, clearly worse than using any contextual hidden layer. Using the single last hidden layer gives 94.9, and the single second-to-last hidden layer gives 95.6. Combining information across the top layers performs best: concatenating the last four hidden layers achieves 96.1 Dev F1 (the best feature-based result, and only 0.3 F1 behind fully fine-tuning BERT_BASE's 96.4), while weighting a sum across all 12 layers gives 95.5, slightly worse than concatenating just the top four. This pattern demonstrates that useful task-relevant information for NER is not concentrated in any single layer, and combining several of the upper layers captures more of it than relying on any one layer alone or on the raw input embeddings.

Q37

Table 8 shows that fine-tuning is surprisingly robust to different masking-rate combinations: MNLI fine-tuning accuracy stays within a narrow band (roughly 83.6–84.4) and NER fine-tuning F1 stays within a narrow band (roughly 94.8–95.4) across all six tested strategies, including the default 80/10/10 strategy. However, the feature-based NER results are more sensitive: using only the MASK strategy (100% MASK, 0% SAME, 0% RND) is described as problematic for the feature-based approach, dropping NER feature-based F1 to 94.0 — the joint lowest score in that column. Using only the RND strategy (0% MASK, 0% SAME, 100% RND) also performs notably worse than the default strategy on MNLI (83.6, the lowest MNLI score in the table).

Q38

The authors conclude from Table 6 that larger models lead to a strict accuracy improvement across all four datasets tested, even for MRPC, which has only 3,600 labeled training examples and is substantially different in nature from the pre-training tasks. They call this "perhaps surprising" because these gains are achieved on top of models that are already quite large relative to the existing literature — for comparison, they note the largest Transformer explored in Vaswani et al. (2017) had about 100M encoder parameters, and the largest Transformer found elsewhere in the literature (Al-Rfou et al., 2018) had about 235M parameters, while BERT_BASE and BERT_LARGE already have 110M and 340M parameters respectively before any of this additional scaling analysis.

Q39

The introduction's claim (page 4171) is that unidirectional constraints could be "very harmful" for token-level tasks like QA. The Table 5 ablation in Section 5.1 (page 4178) provides direct experimental evidence: comparing BERT_BASE (SQuAD F1 88.5) to the "LTR & No NSP" left-to-right model (SQuAD F1 77.8) shows a 10.7-point drop — clearly larger than the drops on the GLUE sentence-level tasks in the same table (e.g., SST-2 drops only 0.6 points, from 92.7 to 92.1). The accompanying text in Section 5.1 explains this directly: it states that a left-to-right model performs poorly at token-level predictions specifically because the token-level hidden states have no right-side context, which is exactly the mechanism the introduction anticipated in the abstract discussion of QA needing bidirectional context.

Q40

Section 3 (page 4173) asserts that BERT's architecture is unified across tasks, with minimal difference between the pre-trained and fine-tuned architectures. Section 3.2 (page 4175) makes this concrete: at the input side, the sentence A/sentence B format used in pre-training is reinterpreted, without architectural change, as sentence pairs in paraphrasing, hypothesis-premise pairs in entailment, question-passage pairs in question answering, or a degenerate text-∅ pair for single-text classification/tagging tasks. At the output side, the same architecture branches only in which output layer is attached: token-level representations feed an output layer for token-level tasks like sequence tagging or QA, while the [CLS] representation feeds an output layer for classification tasks like entailment or sentiment analysis. In every case, the underlying pre-trained Transformer parameters are reused as-is, and only a small, task-specific output layer is newly introduced — directly realizing the "minimal difference" and "unified architecture" claims from Section 3.

Q41

In the same paragraph as the "first work to demonstrate convincingly" claim, the authors cite two contrasting prior findings: Peters et al. (2018b) reported mixed results on downstream task performance when increasing a pre-trained bidirectional LM's size from two to four layers, and Melamud et al. (2016) mentioned in passing that increasing hidden

dimension size from 200 to 600 helped, but increasing further to 1,000 brought no further improvement. Both of those prior studies used a feature-based approach. The BERT authors hypothesize that the key difference is the fine-tuning approach itself: when a model is fine-tuned directly on the downstream task, using only a small number of newly (randomly) initialized parameters, the task-specific model can benefit from the larger, more expressive pre-trained representations even when the downstream task's own training data is very small — an effect that a feature-based approach, which keeps the pre-trained representations frozen and adds a separate task-specific architecture on top, apparently does not exhibit as strongly.

Q42

Section 3.1 (page 4174) states the abstract 80/10/10 rule for masking a chosen token position. The Appendix (page 4183, §A.1) makes this concrete with the example sentence "my dog is hairy," where the random masking procedure selects the 4th token, "hairy." It shows: 80% of the time, the word is replaced with [MASK], yielding "my dog is [MASK]"; 10% of the time, it is replaced with a random word, yielding the example "my dog is apple"; and 10% of the time, the word is kept unchanged, yielding "my dog is hairy" — identical to the original sentence, with the appendix explicitly noting this unchanged case is meant to bias the representation toward the actual observed word.

Q43

The abstract's claim (page 4171) that BERT needs no substantial task-specific architecture changes is borne out concretely by Sections 3.2, 4.2, and 4.4. Section 3.2 (page 4175) states the general principle: for each task, the task-specific inputs and outputs are simply plugged into BERT, and all parameters are fine-tuned end-to-end. Section 4.2 (page 4176) shows this in the SQuAD case: the only new parameters introduced during fine-tuning are a start vector $S \in R^H$ and an end vector $E \in R^H$, used via dot products with the token hidden states to score candidate answer spans. Section 4.4 (page 4178) shows this in the SWAG case: the only task-specific parameter introduced is a single vector whose dot product with the [CLS] representation C produces a score for each of the four candidate continuations, normalized with a softmax. In both cases, only a handful of new parameters (a vector or two) are added on top of the full pre-trained Transformer encoder, with no new layers, attention mechanisms, or task-specific sub-networks — directly and concretely supporting the abstract's claim.

Q44

The paper's evidence for this claim is the Section 5.1 ablation study: the "LTR & No NSP" configuration is explicitly described as being trained to be directly comparable to OpenAI GPT, while still using BERT's own (larger) training dataset, BERT's own input representation, and BERT's own fine-tuning scheme. Because this LTR & No NSP model performs substantially worse than full BERT_BASE across GLUE tasks and SQuAD, the authors argue the remaining gap (bidirectionality plus the two pre-training tasks) accounts for the majority of the empirical improvement over GPT. However, this specific evidence would NOT justify the stronger claim that other differences (BooksCorpus+Wikipedia vs. BooksCorpus alone, 128,000- vs. 32,000-word batch sizes, or BERT's per-task fine-tuning learning rate vs. GPT's fixed 5e-5) contribute nothing at all to BERT's real-world advantage over the actual published GPT system — because the "LTR & No NSP" ablation deliberately keeps BERT's larger corpus, input representation, and fine-tuning scheme fixed while varying only bidirectionality and NSP. It therefore isolates the effect of bidirectionality/pre-training tasks *within an already-GPT-matched-as-closely-as-possible setup*, not the total, cumulative effect of every training difference relative to the real GPT model as originally trained and reported.

Q45

No, this is not established, and the paper's own text argues the opposite of "no benefit." The best-performing BERT_LARGE SQuAD 1.1 system explicitly used modest data augmentation: it was first fine-tuned on TriviaQA before being fine-tuned on SQuAD. The paper reports that without this TriviaQA fine-tuning data, the model's Dev/Test scores drop by 0.1–0.4 F1 (a specific range explicitly given), while still outperforming all existing systems by a wide margin. This shows two things simultaneously: BERT's SQuAD performance does benefit somewhat from external QA data (the drop when it's removed is nonzero), but that benefit is small in magnitude, and BERT_LARGE's core strength does not depend heavily on it. Concluding "BERT gets no benefit at all from external QA data" would over-read the "still outperforms... by a wide margin" framing and ignore the explicitly quantified 0.1–0.4 F1 loss.

Q46

The evidence in Table 8 supports the claim that fine-tuning is "surprisingly robust to different masking strategies" — across all six tested MASK/SAME/RND probability combinations, MNLI fine-tuning accuracy and NER fine-tuning F1

both stay within a fairly narrow range, without collapsing even for combinations quite different from BERT's actual 80/10/10 choice. However, the same section immediately provides a caveat that limits this claim: it states that using only the MASK strategy (100% MASK) is problematic specifically when applying the *feature-based* approach to NER, where performance drops to 94.0 F1, the lowest score in that column. This means the "robustness" finding is explicitly scoped to the fine-tuning setting; it would not be justified to claim, based on this evidence, that BERT's representations are robust to masking-strategy choice under every way of using them (fine-tuning and feature-based alike) — the feature-based NER results show a real, paper-acknowledged exception.

Q47

Problem (p. 4171, §1): Standard language models are unidirectional, restricting pre-training architecture choices and being especially harmful for token-level tasks like QA where bidirectional context matters. Motivation (p. 4171–4174, §1, §3.1): Fine-tuning-based approaches in particular need deep bidirectional representations, but a naive bidirectional conditional LM is impossible to train because each word could indirectly "see itself" through the network. Design (p. 4174, §3.1): Solve this via a Masked Language Model — randomly mask 15% of tokens (with an 80/10/10 sub-procedure to reduce pre-train/fine-tune mismatch) and predict them from context, inspired by the Cloze task; additionally add a Next Sentence Prediction task, since sentence-relationship understanding (needed for QA/NLI) is not directly captured by language modeling alone. Method (p. 4173–4175, §3): Implement this as a multi-layer bidirectional Transformer encoder (BERT_BASE: 110M params; BERT_LARGE: 340M params), pre-trained on BooksCorpus + Wikipedia (3.3B words) with the combined MLM+NSP objective, then fine-tuned per downstream task by adding only a small task-specific output layer. Experiment (p. 4175–4179, §4–§5): Fine-tune and evaluate on GLUE, SQuAD v1.1, SQuAD v2.0, and SWAG (Section 4); run ablations removing NSP, removing bidirectionality (LTR & No NSP), varying model size, and comparing fine-tuning to a feature-based approach on NER (Section 5). Evidence: BERT_BASE and BERT_LARGE outperform all prior systems by substantial margins across all eleven tasks (Tables 1–4); removing NSP hurts MNLI/QNLI/SQuAD; removing bidirectionality (going fully left-to-right) hurts every task, especially the token-level SQuAD task (Table 5); larger models improve performance even on small datasets like MRPC (Table 6); a feature-based use of BERT's representations comes close to, but does not quite match, fine-tuning (Table 7). Conclusion (p. 4179, §6): Rich, unsupervised pre-training — generalized here to deep bidirectional architectures via MLM and NSP — is an integral, empirically well-supported contributor to strong performance across a broad set of NLP tasks, including low-resource ones.

Q48

Table 5's "No NSP" row is exactly this counterfactual (MLM retained, NSP removed) tested against BERT_BASE (both trained). MNLI-m accuracy would fall from 84.4 to 83.9 (–0.5 points), and SQuAD F1 would fall from 88.5 to 87.9 (–0.6 points). The paper's own text describes these effects, along with QNLI's, as NSP "hurting performance significantly" — QNLI shows by far the largest drop (88.4 → 84.9, a 3.5-point fall). However, this effect is clearly not uniform: MRPC would barely move (86.7 → 86.5, –0.2) and SST-2 would barely move (92.7 → 92.6, –0.1). So the evidence supports a real but task-dependent effect: sentence-pair-relationship-sensitive tasks (QNLI, MNLI) and the token-level SQuAD task lose noticeably more than single-sentence-style tasks like SST-2, or MRPC (which is itself a sentence-pair task but shows little change here), and QNLI in particular is far more affected than either MNLI or SQuAD.

Q49

Read in isolation, Section 5.1 (Table 5) shows that, holding architecture size roughly fixed near BERT_BASE, switching the *pre-training objective* from bidirectional MLM+NSP to a left-to-right LM without NSP causes large performance drops (e.g., MRPC 86.7→77.5, SQuAD F1 88.5→77.8). Read in isolation, Section 5.2 (Table 6) shows that, holding the pre-training *objective* fixed (always MLM+NSP), increasing model *size* from 3 layers to 24 layers causes strict, continual improvements on MNLI-m, MRPC, and SST-2. Interpreted together rather than separately, these two ablations indicate that BERT's overall strong performance is not attributable to a single cause — it depends on getting the pre-training *objective* right (deep bidirectionality plus NSP, per 5.1) AND on sufficient model *scale* (per 5.2); the two effects appear largely additive and independent factors uncovered by two different, orthogonal experimental manipulations (objective type vs. parameter count), rather than the same underlying variable measured twice. A reading of either ablation alone would risk over-attributing BERT's success either purely to its novel pre-training objective or purely to scale, when the paper's own evidence points to both mattering substantially.

Q50

Section 5.3 lists two advantages of the feature-based approach over fine-tuning. The relevant one here is explicitly computational: there are major computational benefits to pre-computing an expensive representation of the training data once, and then running many experiments with cheaper models on top of that fixed representation. If the authors always used the fine-tuning approach instead, this advantage would be lost — since fine-tuning requires jointly training all of BERT's parameters end-to-end for each downstream task/configuration, every new experiment or model variant would require re-running the full, comparatively expensive fine-tuning process on the entire BERT model, rather than cheaply reusing a single pre-computed set of contextual features across many lightweight downstream experiments.

Q51

"Insufficient information in the paper." The paper reports that BERT_LARGE was trained on 16 Cloud TPUs (64 TPU chips total) over 4 days, but it does not report carbon emissions, energy consumption, or any related environmental-cost figure anywhere in the text.

Q52

"Insufficient information in the paper." All pre-training data described (BooksCorpus and English Wikipedia) and all evaluation benchmarks described (GLUE, SQuAD v1.1, SQuAD v2.0, SWAG, CoNLL-2003 NER) are English-language tasks; the paper contains no experiments, results, or discussion of non-English or multilingual performance.

Q53

"Insufficient information in the paper." Section 4.3 states that a non-null answer is predicted when the best span's score exceeds the no-answer score plus a threshold τ, and that τ "is selected on the dev set to maximize F1" — but the paper never states what numeric value τ actually took, nor does it report any sensitivity analysis of performance with respect to different τ values.

Q54

"Insufficient information in the paper." The paper reports pre-training hardware/duration (TPU counts, days) and fine-tuning training time (e.g., roughly 30 minutes on a single Cloud TPU for the SQuAD model, or at most an hour on a single Cloud TPU for GLUE tasks), but it does not report or compare inference/prediction latency for BERT versus ELMo, or versus any other system, anywhere in the paper.

Q55

"Insufficient information in the paper." Appendix A.4 states that GPT was trained with a batch size of 32,000 words while BERT was trained with 128,000 words, as one of several listed differences between the two systems' training setups. However, the paper does not report any ablation experiment that isolates the effect of batch size alone (holding everything else constant) on GLUE or any other downstream score — the only ablations reported (Section 5) vary the pre-training objective/bidirectionality and the model size, not the pre-training batch size. Any specific GLUE score under this counterfactual batch size is therefore not something the paper provides.